Table of contents

Boolean functions

Last update: October 11, 2024

Bookmark Copy link

TOPICS: Segments

CREATED FOR: User

Boolean functions are used to perform boolean logic on different elements in Profile Query Language (PQL). More information about other PQL functions can be found in the Profile Query Language overview.

And

The `and` function is used to create a logical conjunction as a boolean.

Format

```
{QUERY} and {QUERY}
```

Example

The following PQL query will return all people with home country as Canada and birth year of 1985.

```
homeAddress.countryISO = "CA" and person.birthYear = 1985
```

Or

The `or` function is used to create a logical disjunction as a boolean.

Was this content helpful?

Detailed feedback options

```
{QUERY} or {QUERY}
```

Example

The following PQL query will return all people with home country as Canada or birth year of 1985.

```
homeAddress.countryISO = "CA" or person.birthYear = 1985
```

Not

The `not` (or `!`) function is used to create a logical negation.

Format

```
not ({QUERY})
!({QUERY})
```

Example

The following PQL query will return all people who do not have their home country as Canada.

```
not (homeAddress.countryISO = "CA")
```

If

The `if` function is used to resolve an expression depending on whether a specified condition is true as a boolean.

Format

```
if ({TEST_EXPRESSION}, {TRUE_EXPRESSION}, {FALSE_EXPRESSION})
```

Argument	Description

Was this content helpful?

Detailed feedback options

Argument	Description
`{TRUE_EXPRESSION}`	The expression whose value will be used if `{TEST_EXPRESSION}` is true.
`{FALSE_EXPRESSION}`	The expression whose value will be used if `{TEST_EXPRESSION}` is false.

Example

The following PQL query will set the value as `1` if the home country is Canada and `2` if the home country is not Canada.

```
if (homeAddress.countryISO = "CA", 1, 2)
```

Next steps

Now that you have learned about boolean functions, you can use them within your PQL queries. For more information about other PQL functions, please read the Profile Query Language overview.

More help on this topic

- Comparison functions
- Array, list, and set functions
- Filter functions

Previous page
OVERVIEW

Next page
COMPARISON FUNCTIONS

Was this content helpful?

Detailed feedback options

Table of contents

Comparison functions

Last update: October 10, 2024

Bookmark Copy link

TOPICS: Segments

CREATED FOR: User

Comparison functions are used to compare between different expressions and values, returning `true` or `false` accordingly. More information about other PQL functions can be found in the Profile Query Language overview.

Equals

The = (equals) function checks whether one value or expression is equal to another value or expression.

Format

```
{EXPRESSION} = {VALUE}
```

Example

The following PQL query checks if the home address country is in Canada.

```
homeAddress.countryISO = "CA"
```

Not equal

The `!=` (not equal) function checks whether one value or expression is **not** equal to another value or expression.

Format

```
{EXPRESSION} != {VALUE}
```

Example

The following PQL query checks if the home address country is not in Canada.

```
homeAddress.countryISO != "CA"
```

Greater than

The > (greater than) function is used to check if the first value is greater than the second value.

Format

```
{EXPRESSION} > {EXPRESSION}
```

Example

The following PQL query defines people whose birthdays do not fall in January or February.

```
person.birthMonth > 2
```

Greater than or equal to

The >= (greater than or equal to) function is used to check if the first value is greater than or equal to the second value.

Format

```
{EXPRESSION} >= {EXPRESSION}
```

Example

The following PQL query defines people whose birthdays do not fall in January or February.

```
person.birthMonth >= 3
```

Less than

The < (less than) comparison function is used to check if the first value is less than the second value.

Format

```
{EXPRESSION} < {EXPRESSION}
```

Example

The following PQL query defines people whose birthday is in January.

```
person.birthMonth < 2
```

Less than or equal to

The <= (less than or equal to) comparison function is used to check if the first value is less than or equal to the second value.

Format

```
{EXPRESSION} <= {EXPRESSION}
```

Example

The following PQL query defines people whose birthday is in January or February.

```
person.birthMonth <= 2
```

Next steps

Now that you have learned about comparison functions, you can use them within your PQL queries. For more information about other PQL functions, please read the [Profile Query Language overview](#).

More help on this topic

- [Array, list, and set functions](#)

- Boolean functions
- Date and time functions

Previous page
BOOLEAN FUNCTIONS

Next page
ARRAY, LIST, AND SET FUNCTIONS

Table of contents

Array, list, and set functions

Last update: April 1, 2026

Bookmark Copy link

TOPICS: Segments

CREATED FOR: User

Profile Query Language (PQL) offers functions to make interaction with arrays, lists, and strings easier. More information about other PQL functions can be found in the Profile Query Language overview.

In

The `in` function is used to determine if an item is a member of an array or list as a boolean.

Format

```
{VALUE} in {ARRAY}
```

Example

The following PQL query defines people with birthdays in March, June, or September.

```
person.birthMonth in [3, 6, 9]
```

Not in

The `notIn` function is used to determine if an item is not a member of an array or list as a boolean.

> **NOTE**
>
> The `notIn` function *also* ensures that neither value is equal to null. Therefore, the results are not an exact negation of the `in` function.

Format

```
{VAL
```

Was this content helpful?

Detailed feedback options

Example

The following PQL query defines people with birthdays that are not in March, June, or September.

```
person.birthMonth notIn [3, 6, 9]
```

Intersects

The `intersects` function is used to determine if two arrays or lists have at least one common member as a boolean.

Format

```
{ARRAY}.intersects({ARRAY})
```

Example

The following PQL query defines people whose favorite colors include at least one of red, blue, or green.

```
person.favoriteColors.intersects(["red", "blue", "green"])
```

Intersection

The `intersection` function is used to determine the common members of two arrays or lists as a list.

Format

```
{ARRAY}.intersection({ARRAY})
```

Example

The following PQL query defines if person 1 and person 2 both have favorite colors of red, blue, and green.

```
person1.favoriteColors.intersection(person2.favoriteColors) = ["red", "blue", "green"]
```

Subset of

The `subsetOf` function is used to determine if a specific array (array A) is a subset of another array (array B). In other words, that all elements in array A are elements of array B as a boolean.

Format

```
{ARR
```

Was this content helpful?

Detailed feedback options

Exampl

The following PQL query defines people who have visited all of their favorite cities.

```
person.favoriteCities.subsetOf(person.visitedCities)
```

Superset of

The `supersetOf` function is used to determine if a specific array (array A) is a superset of another array (array B). In other words, that array A contains all elements in array B as a boolean.

Format

```
{ARRAY}.supersetOf({ARRAY})
```

Example

The following PQL query defines people who have eaten sushi and pizza at least once.

```
person.eatenFoods.supersetOf(["sushi", "pizza"])
```

Includes

The `includes` function is used to determine if an array or list contains a given item as a boolean.

Format

```
{ARRAY}.includes({ITEM})
```

Example

The following PQL query defines people whose favorite color includes red.

```
person.favoriteColors.includes("red")
```

Distinct

The `distinct` function is used to remove duplicate values from an array or list as an array.

Format

```
{ARRAY}.distinct()
```

Exampl

The foll

Was this content helpful?

Detailed feedback options

```
person.orders.storeId.distinct().count() > 1
```

Group by

The `groupBy` function is used to partition values of an array or list into a group based on the value of the expression as a map from unique values of the grouping expression to arrays which are partitions of the value of the array expression.

Format

```
{ARRAY}.groupBy({EXPRESSION})
```

Argument	Description
`{ARRAY}`	The array or list that is to be grouped.
`{EXPRESSION}`	An expression which maps each item in the array or list returned.

Example

The following PQL query groups all the orders by which store the order was placed at.

```
xEvent[type="order"].groupBy(storeId)
```

Filter

The `filter` function is used to filter an array or list based on an expression as an array or list, depending on the input.

Format

```
{ARRAY}.filter({EXPRESSION})
```

Argument	Description
`{ARRAY}`	The array or list that is to be filtered.
`{EXPRESSION}`	An expression to filter by.

Was this content helpful?

Detailed feedback options

Exampl

The following PQL query defines all people who are 21 or older.

```
person.filter(age >= 21)
```

Map

The `map` function is used to create a new array by applying an expression to each item in a given array as an array.

Format

```
array.map(expression)
```

Example

The following PQL query creates a new array of numbers and squares the value of the original numbers.

```
numbers.map(square)
```

First n in array

The `topN` function is used to return the first `N` items in an array, when sorted in ascending order based on the given numerical expression as an array.

Format

```
{ARRAY}.topN({VALUE}, {AMOUNT})
```

Argument	Description
`{ARRAY}`	The array or list that is to be sorted.
`{VALUE}`	The property in which to sort the array or list.
`{AMOUNT}`	The number of items to be returned.

Example

The following PQL query returns the top five orders with the highest price.

```
orders.topN(price, 5)
```

Was this content helpful?

Detailed feedback options

Last r

The `bottomN` function is used to return the last `N` items in an array, when sorted in ascending order based on the given numerical expression as an array.

Format

```
{ARRAY}.bottomN({VALUE}, {AMOUNT})
```

Argument	Description
`{ARRAY}`	The array or list that is to be sorted.
`{VALUE}`	The property in which to sort the array or list.
`{AMOUNT}`	The number of items to be returned.

Example

The following PQL query returns the top five orders with the lowest price.

```
orders.bottomN(price, 5)
```

First item

The `head` function is used to return the first item in the array or list as an object.

Format

```
{ARRAY}.head()
```

Example

The following PQL query returns the first of the top five orders with the highest price. More information about the `topN` function can be found in the first n in array section.

```
orders.topN(price, 5).head()
```

Next steps

Now that you have learned about array, list, and set functions, you can use them within your PQL queries. For more information about other PQL functions, please read the Profile Query Language overview.

More Was this content helpful?

Detailed feedback options

- Comparison functions
- Boolean functions
- Filter functions

Previous page
COMPARISON FUNCTIONS

Next page
MAP FUNCTIONS

Was this content helpful?

Detailed feedback options

Table of contents

Map functions

Last update: October 11, 2024

Bookmark Copy link

TOPICS: Segments

CREATED FOR: User

Profile Query Language (PQL) offers functions to make interaction with maps easier. More information about other PQL functions can be found in the Profile Query Language overview.

Get

The `get` function is used to retrieve the value of a map for a given key as an object.

Format

```
{MAP}.get({STRING})
```

Example

The following PQL query gets the value of the identity map for the key `example@example.com`.

```
identityMap.get("example@example.com")
```

Keys

The `keys` function is used to retrieve all the keys for a given map as an array or list.

Format

Was this content helpful?

Detailed feedback options

Example

The following PQL query gets all the keys for the map `identityMap`.

```
identityMap.keys()
```

Values

The `values` function is used to retrieve all the values of a given map as an array or list.

Format

```
{MAP}.values()
```

Example

The following PQL query gets all the values for the map `identityMap`.

```
identityMap.values()
```

Next steps

Now that you have learned about map functions, you can use them within your PQL queries. For more information about other PQL functions, please read the Profile Query Language overview.

More help on this topic

- String functions
- Comparison functions
- Array, list, and set functions

Previous page
ARRAY, LIST, AND SET FUNCTIONS

Next page
STRING FUNCTIONS

Was this content helpful?

Detailed feedback options

Was this content helpful?

Detailed feedback options

Table of contents

String functions

Last update: October 11, 2024

Bookmark Copy link

TOPICS: Segments

CREATED FOR: User

Profile Query Language (PQL) offers functions to make interaction with strings simpler. More information about other PQL functions can be found in the Profile Query Language overview.

Like

The `like` function is used to determine if a string matches a specified pattern as a boolean.

Format

```
{STRING_1} like {STRING_2}
```

Argument	Description
`{STRING_1}`	The string to perform the check on.
`{STRING_2}`	The expression to match against the first string. There are two supported special characters for creating an expression: `%` and `_`. • `%` is used to represent zero or more characters. • is used to represent exactly one

Was this content helpful?

Detailed feedback options

Example

The following PQL query retrieves all the cities containing the pattern “es”.

```
city like "%es%"
```

Starts with

The `startsWith` function is used to determine if a string starts with a specified substring as a boolean.

Format

```
{STRING_1}.startsWith({STRING_2}, {BOOLEAN})
```

Argument	Description
`{STRING_1}`	The string to perform the check on.
`{STRING_2}`	The string to search for within the first string.
`{BOOLEAN}`	An optional parameter to determine if the check is case sensitive. By default, this is set to true.

Example

The following PQL query determines, with case sensitivity, if the person’s name starts with “Joe”.

```
person.name.startsWith("Joe")
```

Does not start with

The `doesNotStartWith` function is used to determine if a string does not start with a specified

Was this content helpful?

Detailed feedback options

```
{STRING_1}.doesNotStartWith({STRING_2}, {BOOLEAN})
```

Argument	Description
`{STRING_1}`	The string to perform the check on.
`{STRING_2}`	The string to search for within the first string.
`{BOOLEAN}`	An optional parameter to determine if the check is case sensitive. By default, this is set to true.

Example

The following PQL query determines, with case sensitivity, if the person's name does not start with "Joe".

```
person.name.doesNotStartWith("Joe")
```

Ends with

The `endsWith` function is used to determine if a string ends with a specified substring as a boolean.

Format

```
{STRING_1}.endsWith({STRING_2}, {BOOLEAN})
```

Argument	Description
`{STRING_1}`	The string to perform the check on.
`{STRING_2}`	The string to search for within the first

Was this content helpful?

Detailed feedback options

Argument	Description
`{BOOLEAN}`	An optional parameter to determine if the check is case sensitive. By default, this is set to true.

Example

The following PQL query determines, with case sensitivity, if the person's email address ends with ".com".

```
person.emailAddress.endsWith(".com")
```

Does not end with

The `doesNotEndWith` function is used to determine if a string does not end with a specified substring as a boolean.

Format

```
{STRING_1}.doesNotEndWith({STRING_2}, {BOOLEAN})
```

Argument	Description
`{STRING_1}`	The string to perform the check on.
`{STRING_2}`	The string to search for within the first string.
`{BOOLEAN}`	An optional parameter to determine if the check is case sensitive. By default, this is set to true.

Example

Was this content helpful?

Detailed feedback options

```
person.emailAddress.doesNotEndWith(".com")
```

Contains

The `contains` function is used to determine if a string contains a specified substring as a boolean.

Format

```
{STRING_1}.contains({STRING_2}, {BOOLEAN})
```

Argument	Description
`{STRING_1}`	The string to perform the check on.
`{STRING_2}`	The string to search for within the first string.
`{BOOLEAN}`	An optional parameter to determine if the check is case sensitive. By default, this is set to true.

Example

The following PQL query determines, with case sensitivity, if the person's email address contains the string "2010@gm".

```
person.emailAddress.contains("2010@gm")
```

Does not contain

The `doesNotContain` function is used to determine if a string does not contain a specified substring as a boolean.

Format

Was this content helpful?

Detailed feedback options

Argument	Description
`{STRING_1}`	The string to perform the check on.
`{STRING_2}`	The string to search for within the first string.
`{BOOLEAN}`	An optional parameter to determine if the check is case sensitive. By default, this is set to true.

Example

The following PQL query determines, with case sensitivity, if the person's email address does not contain the string "2010@gm".

```
person.emailAddress.doesNotContain("2010@gm")
```

Equals

The `equals` function is used to determine if a string is equal to the specified string as a boolean.

Format

```
{STRING_1}.equals({STRING_2})
```

Argument	Description
`{STRING_1}`	The string to perform the check on.
`{STRING_2}`	The string to compare with the first string.

Example

Was this content helpful?

Detailed feedback options

```
person.name.equals("John")
```

Not equal to

The `notEqualTo` function is used to determine if a string is not equal to the specified string as a boolean.

Format

```
{STRING_1}.notEqualTo({STRING_2})
```

Argument	Description
`{STRING_1}`	The string to perform the check on.
`{STRING_2}`	The string to compare with the first string.

Example

The following PQL query determines, with case sensitivity, if the person's name is not "John".

```
person.name.notEqualTo("John")
```

Matches

The `matches` function is used to determine if a string matches a specific regular expression. Please refer to this document for more information on matching patterns in regular expressions as a boolean.

Format

```
{STRING_1}.matches(STRING_2})
```

Example

Was this content helpful?

Detailed feedback options

```
person.name.matches("(?i)^John")
```

> **NOTE**
>
> If you are using regular expression functions such as `\w`, you **must** escape the backslash character. So, instead of writing just `\w`, you must include an extra backslash and write `\\w`.

Regular expression group

The `regexGroup` function is used to extract specific information, based on the regular expression provided as a string.

Format

```
{STRING}.regexGroup({EXPRESSION})
```

Example

The following PQL query is used to extract the domain name from an email address.

```
emailAddress.regexGroup("@(\\w+)", 1)
```

> **NOTE**
>
> If you are using regular expression functions such as `\w`, you **must** escape the backslash character. So, instead of writing just `\w`, you must include an extra backslash and write `\\w`.

Next steps

Now that you have learned about string functions, you can use them within your PQL queries. For more information about other PQL functions, please read the [Profile Query Language overview](#).

Was this content helpful?

Detailed feedback options

- Comparison functions
- Boolean functions

Previous page
MAP FUNCTIONS

Next page
OBJECT FUNCTIONS

Was this content helpful?

Detailed feedback options

Table of contents

Object functions

Last update: October 11, 2024

Bookmark Copy link

TOPICS: Segments

CREATED FOR: User

Profile Query Language (PQL) offers functions to make interaction with objects simpler. More information about other PQL functions can be found in the Profile Query Language overview.

Is null

The `isNull` function determines if an object reference does not exist as a boolean.

Format

```
{OBJECT}.isNull()
```

Example

The following PQL query checks if the person's home address does not exist.

```
person.homeAddress.isNull()
```

Is not null

The `isNotNull` function determines if an object reference exists as a boolean.

Format

Was this content helpful?

Detailed feedback options

Example

The following PQL query checks if the person's home address exists.

```
person.homeAddress.isNotNull()
```

Next steps

Now that you have learned about object functions, you can use them within your PQL queries. For more information about other PQL functions, please read the Profile Query Language overview.

More help on this topic

- Filter functions
- Comparison functions
- Array, list, and set functions

Previous page
STRING FUNCTIONS

Next page
ARITHMETIC FUNCTIONS

Was this content helpful?

Detailed feedback options

Table of contents

Arithmetic functions

Last update: October 11, 2024

Bookmark Copy link

TOPICS: Segments

CREATED FOR: User

Arithmetic functions are used to perform basic calculations on values in Profile Query Language (PQL). More information about other PQL functions can be found in the Profile Query Language overview.

Add

The + (addition) function is used to find the sum of two argument expressions as a number.

Format

```
{NUMBER} + {NUMBER}
```

Example

The following PQL query sums the price of two different products.

```
product1.price + product2.price
```

Multiply

The * (multiplication) function is used to find the product of two argument expressions as a number.

Was this content helpful?

Detailed feedback options

```
{NUMBER} * {NUMBER}
```

Example

The following PQL query finds the product of the inventory and the price of a product to find the gross value of the product.

```
product.inventory * product.price
```

Subtract

The `-` (subtraction) function is used to find the difference of two argument expressions as a number.

Format

```
{NUMBER} - {NUMBER}
```

Example

The following PQL query finds the difference in price between two different products.

```
product1.price - product2.price
```

Divide

The `/` (division) function is used to find the quotient of two argument expressions as a number.

Format

```
{NUMBER} / {NUMBER}
```

Example

The following PQL query finds the quotient between the total products sold and total money earned to see the average cost per item.

Was this content helpful?

Detailed feedback options

Remainder

The `%` (modulo/remainder) function is used to find the remainder after dividing the two argument expressions as a number.

Format

```
{NUMBER} % {NUMBER}
```

Example

The following PQL query checks if the person's age is divisible by five.

```
person.age % 5 = 0
```

Next steps

Now that you have learned about arithmetic functions, you can use them within your PQL queries. For more information about other PQL functions, please read the Profile Query Language overview.

More help on this topic

- Comparison functions
- Date and time functions
- Object functions

Previous page
OBJECT FUNCTIONS

Next page
AGGREGATION FUNCTIONS

Was this content helpful?

Detailed feedback options

Table of contents

Aggregation functions

Last update: October 11, 2024

Bookmark Copy link

TOPICS: Segments

CREATED FOR: User

Aggregation functions are used to group together multiple values within Profile Query Language (PQL) arrays to form a single summary value. More information about other PQL functions can be found in the Profile Query Language overview.

Count

The `count` function returns the number of elements within the given array as a number.

Format

```
{ARRAY}.count()
```

Example

The following PQL query returns the number of orders in the array.

```
orders.count()
```

Sum

The `sum` function returns the sum of all the selected values within the array as a number.

Format

Was this content helpful?

Detailed feedback options

Example

The following PQL query returns the sum of all the orders' prices.

```
orders.sum(order.price)
```

Average

The `average` function returns the arithmetic mean of all the selected values within the array as a number.

Format

```
{ARRAY}.average()
```

Example

The following PQL query returns the average price of all the orders.

```
orders.average(order.price)
```

Minimum

The `min` function returns the smallest of all the selected values within the array as a number.

Format

```
{ARRAY}.min()
```

Example

The following PQL query returns the lowest price of all the orders.

```
orders.min(order.price)
```

Maximum

Was this content helpful?

Detailed feedback options

Format

```
{ARRAY}.max()
```

Example

The following PQL query returns the highest price of all the orders.

```
orders.max(order.price)
```

Next steps

Now that you have learned about aggregation functions, you can use them within your PQL queries. For more information about other PQL functions, please read the Profile Query Language overview.

More help on this topic

- Segmentation Service overview
- Streaming segmentation guide
- Map functions

Previous page
ARITHMETIC FUNCTIONS

Next page
DATE AND TIME FUNCTIONS

Was this content helpful?

Detailed feedback options

Table of contents

Date and time functions

Last update: October 11, 2024

Bookmark Copy link

TOPICS: Segments

CREATED FOR: User

Date and time functions are used to perform date and time operations on values within Profile Query Language (PQL). More information about other PQL functions can be found in the Profile Query Language overview.

Current month

The `currentMonth` function returns the current month as an integer.

Format

```
currentMonth()
```

Example

The following PQL query checks if the person's birth month is the current month.

```
person.birthMonth = currentMonth()
```

Get month

The `getMonth` function returns the month, as an integer, based on a given timestamp.

Format

Was this content helpful?

Detailed feedback options

Example

The following PQL query checks if the person's birth month is in June.

```
person.birthdate.getMonth() = 6
```

Current year

The `currentYear` function returns the current year as an integer.

Format

```
currentYear()
```

Example

The following PQL query checks if the product was sold in the current year.

```
product.saleYear = currentYear()
```

Get year

The `getYear` function returns the year, as an integer, based on a given timestamp.

Format

```
{TIMESTAMP}.getYear()
```

Example

The following PQL query checks if the person's birth year falls in 1991, 1992, 1993, 1994, or 1995.

```
person.birthday.getYear() in [1991, 1992, 1993, 1994, 1995]
```

Current day of month

Was this content helpful?

Detailed feedback options

```
currentDayOfMonth()
```

Example

The following PQL query checks if the person's birth day matches the current day of the month.

```
person.birthDay = currentDayOfMonth()
```

Get day of month

The `getDayOfMonth` function returns the day, as an integer, based on a given timestamp.

Format

```
{TIMESTAMP}.getDayOfMonth()
```

Example

The following PQL query checks if the item was sold within the first 15 days of the month.

```
product.sale.getDayOfMonth() <= 15
```

Occurs

The `occurs` function compares the given timestamp function with a fixed period of time as a boolean.

Format

The `occurs` function can be written using any of the following formats:

```
{TIMESTAMP} occurs {COMPARISON} {INTEGER} {TIME_UNIT} {DIRECTION} {TIME}
{TIMESTAMP} occurs {DIRECTION} {TIME}
{TIMESTAMP} occurs (on) {TIME}
{TIMESTAMP} occurs between {TIME} and {TIME}
```

Was this content helpful?

Detailed feedback options

Argument	Description
`{COMPARISON}`	A comparison operator. Can be any of the following operators: >, >=, <, <=, =, !=. More information about the comparison functions can be found in the comparison functions document.
`{INTEGER}`	A non-negative integer.
`{TIME_UNIT}`	A unit of time. Can be any of the following words: `millisecond(s)`, `second(s)`, `minute(s)`, `hour(s)`, `day(s)`, `week(s)`, `month(s)`, `year(s)`, `decade(s)`, `century`, `centuries`, `millennium`, `millennia`.
`{DIRECTION}`	A preposition describing when to compare the date to. Can be any of the following words: `before`, `after`, `from`.
`{TIME}`	Can be a timestamp literal (`today`, `now`, `yesterday`, `tomorrow`), a relative time unit (one of `this`, `last`, or `next` followed by a time unit), or a timestamp attribute.

> **NOTE**
>
> Usage of the word `on` is optional. It is there to improve readability for some combinations, such as `timestamp occurs on date(2019,12,31)`.

Example

The following PQL query checks if the item was sold last week.

```
product.saleDate occurs last week
```

Was this content helpful?

Detailed feedback options

```
product.saleDate occurs between date(2015, 1, 8) and date(2017, 7, 1)
```

Now

`now` is a reserved word that represents the timestamp of the PQL execution.

Example

The following PQL query checks if an item was sold exactly three hours before.

```
product.saleDate occurs = 3 hours before now
```

Today

`today` is a reserved word that represents the timestamp of the start of the day of the PQL execution.

Example

The following PQL query checks if a person's birthday was three days ago.

```
person.birthday occurs = 3 days before today
```

Next steps

Now that you have learned about date and time functions, you can use them within your PQL queries. For more information about other PQL functions, please read the [Profile Query Language overview](#).

More help on this topic

- [Filter functions](#)
- [Comparison functions](#)
- Arithmetic functions

Was this content helpful?

Detailed feedback options

Previous page

AGGREGATION FUNCTIONS

Next page

FILTER FUNCTIONS

Was this content helpful?

Detailed feedback options

Table of contents

Filter functions

Last update: October 11, 2024

Bookmark Copy link

TOPICS: Segments

CREATED FOR: User

Filter functions are used to filter data within arrays in Profile Query Language (PQL). More information about other PQL functions can be found in the Profile Query Language overview.

Filter

The `[]` (filter) function allows filters to be applied to an array and return a subset of the array which match the specified condition. As a result, this function returns an array.

Format

```
{ARRAY}[filter]
```

Example

The following PQL query gets all events which have at least one product item with an SKU equal to "PS".

```
xEvent[productListItems[SKU="PS"]]
```

Up operator

The ^ (up) operator allows you to refer to properties in upper levels of filters.

Was this content helpful?

Detailed feedback options

```
{ARRAY}[{FILTER_1}[{FILTER_2} or ^{PROPERTY}]]
```

Argument	Description
{ARRAY}	The array that is being filtered.
{FILTER_1}	The outer layer of filtering.
{FILTER_2}	The inner layer of filtering
^{PROPERTY}	The property that is also being filtered on. Due to the ^, it is checking a property based on filter1.

Example

The following PQL query gets all events which have at least one product item with an SKU equal to "PS" **or** have a person whose gender is female.

```
xEvent[productListItems[SKU="PS" or ^^.person.gender="female"]]
```

Next steps

Now that you have learned about filter functions, you can use them within your PQL queries. For more information about other PQL functions, please read the Profile Query Language overview.

More help on this topic

- Date and time functions
- Comparison functions
- Object functions

Was this content helpful?

Detailed feedback options

Was this content helpful?

Detailed feedback options

Table of contents

Logical quantifier functions

Last update: April 1, 2026

Bookmark Copy link

TOPICS: Segments

CREATED FOR: User

Logical quantifiers can be used to assert conditions with arrays in Profile Query Language (PQL). More information about other PQL functions can be found in the Profile Query Language overview.

Exists

The `exists` function determines the existence of an item in an array, provided it satisfies the provided condition.

Format

```
exists {VARIABLE} from {EXPRESSION} where {CONDITION}
exists {VARIABLE} from {EXPRESSION} : {CONDITION}
```

Argument	Description
`{VARIABLE}`	A name of a variable.
`{EXPRESSION}`	The array which is being checked.
`{CONDITION}`	An optional expression which filters the values in the array returned.

Example

The following PQL query gets all events which has a price greater than $50 or has a SKU of “PS”.

```
exists E from xEvent where (E.commerce.item.price > 50), I from E.productListItems where I.SKU = "PS"
```

For all

The `forall` function determines all items in an array that satisfy all the given conditions.

Format

```
forall {VARIABLE} from {EXPRESSION} where {CONDITION}
forall {VARIABLE} from {EXPRESSION} : {CONDITION}
```

Was this content helpful?

Detailed feedback options

Argument	Description
`{VARIABLE}`	A name of a variable.
`{EXPRESSION}`	The array which is being checked.
`{CONDITION}`	An optional expression which filters the values in the array returned.

Example

The following PQL query gets all events which has a price greater than $50 and has a SKU of "PS".

```
forall E from xEvent where (E.commerce.item.price > 50), I from E.productListItems where I.SKU = "PS"
```

Next steps

Now that you have learned about logical quantifiers, you can use them within your PQL queries. For more information about other PQL functions, please read the Profile Query Language overview.

More help on this topic

- Miscellaneous functions
- Profile Query Language (PQL) overview
- Supported data types in Segmentation Service

Previous page
FILTER FUNCTIONS

Next page
MISCELLANEOUS FUNCTIONS

Was this content helpful?

Detailed feedback options

Table of contents

Miscellaneous functions

Last update: June 19, 2023

Bookmark Copy link

TOPICS: Segments

CREATED FOR: User

The following function is a miscellaneous function for Profile Query Language (PQL). More information about other PQL functions can be found in the Profile Query Language overview.

Let

The `let` function allows an expression to be stored as a variable to be used later in a query.

Format

```
let {VARIABLE} = {EXPRESSION}
```

Example

The following PQL query gets all sums of product totals with the transaction in USD where the sum is greater than $100 and less than $1000.

```
let S = (sum X.commerce.order.priceTotal over X from xEvent where X.commerce.order.currencyCode = "USD") in (S > 10
```

Next steps

Now that you have learned about miscellaneous functions, you can use them within your PQL queries. For more information about other PQL functions, please read the Profile Query Language overview.

More help on this topic

- Logical quantifier functions
- Profile Query Language (PQL) overview
- Supported data types in Segmentation Service

Previous page
LOGICAL QUANTIFIERS

Next page
FREQUENTLY ASKED QUESTIONS

Was this content helpful?

Detailed feedback options

Table of contents

Profile Query Language (PQL) overview

Last update: April 1, 2026

Bookmark Copy link

TOPICS: Segments

CREATED FOR: User

Profile Query Language (PQL) is an Experience Data Model (XDM) compliant query language which is designed to support the definition and execution of segmentation queries for Real-Time Customer Profile data.

This guide provides a general overview of PQL, covering formatting guidelines and providing example PQL expressions.

PQL query formatting

PQL queries have the following signature:

```
({INPUT_PARAMETER_1}, {INPUT_PARAMETER_2}, ...) => {RESULT_TYPE}
```

The input parameter can be a simple primitive, such as a boolean or a string, or a more complex type, such as an object, array, or map.

There are three different ways to refer to input parameters within the body of a PQL expression:

Implicit reference to the first parameter

In the example below, since the first parameter is always in context, a property reference (`homeAddress`) can be made directly to it.

```
homeAddress.stateProvince = workAddress.stateProvince
```

Explicit reference to the first parameter

In the example below, `$1` refers to the first parameter. A re `$2` would refer to the second parameter, etc.

```
$1.homeAddress.stateProvince = $1.homeAddress.stateProvince
```

Usage of named variables, using the lambda notation

In the example below, `Profile` is a variable name, which can be chosen by the query author.

```
(Profile) => Profile.homeAddress.stateProvince = Profile.workAddress.stateProvince
```

PQL literals

PQL provides support for the following literal types:

Literal	Definition	Example
String	A data type comprised of characters surrounded by double quotes.	`"pizza"`, `"jobs"`, `"antidisestablishmentarianism"`
Boolean	A data type that is either true or false.	`true`, `false`
Integer	A data type representing a whole number. It can be positive, negative, or zero.	`-201`, `0`, `412`
Double	A data type representing any real number. It can be positive, negative, or zero.	`-51.24`, `3.14`, `0.6942058`
Date	A data type that can be used to create dates based on the year, month, and day as integer parameters. It is formatted as `date(year, month, day)`	`date(2020, 3, 14)`

Literal	Definition	Example
Array	A data type that is comprised as a group of other literal values. It uses square brackets to group and commas to delimit between different values. **Note:** You cannot directly access properties of items within an array. So, if you need to access a property within an array, the supported method is `select X from array where X.item = ....` PQL reserves the word `xEvent` to refer to an array of experience events linked to a profile.	`[1, 4, 7]`, `["US", "CA"]`
Relative time references	Reserved words that can be used to form timestamp and time interval references. • now, today, yesterday, tomorrow • this, last, next • before, after, from • millisecond(s), second(s), minute(s), hour(s), day(s), week(s), month(s), year(s), decade(s), century/centuries, millennium/millennia	`X.timestamp occurs before today`, `X.timestamp occurs last month`, `X.timestamp occurs <= 3 days before now`

PQL functions

The following table outlines the different categories of supported PQL functions, including links to further documentation for more information.

Category	Definition
Boolean	Used to implement boolean algebra within PQL. More information about these functions can be found in the boolean functions document.
Comparison	Used to compare between different PQL elements. More information about these functions can be found in the comparison functions document.
Array, list, and set	Used to interact with arrays, lists, and sets. More information about these functions can be found in the array, list, and set functions document.
Map	Used to interact with maps. More information about these functions can be found in the map functions document.
String	Used to interact with strings. More information about these functions can be found in the string functions document.
Object	Used to interact with objects. More information about these functions can be found in the object functions document.
Arithmetic	Used to perform basic arithmetic on PQL elements. More information about these functions can be found in the arithmetic functions document
Aggregation	Used to combine results of an array into a singular result. More information about aggregation functions can be found in the aggregation functions document.
Date and time	Used in conjunction with date, time, and datetime objects. More information about these functions can be found in the date/time functions document.
Filter	Ued to filter data within arrays. More information about these functions can be found in the filter functions document.

Category	Definition
Logical quantifiers	Used to assert conditions within an array. More information can be found in the logical quantifiers document.
Miscellaneous	Functions that do not fit in any of the above categories can be found in the miscellaneous functions document.

Next steps

Now that you've learned how to use Profile Query Language, you can use PQL when creating and modifying segment definitions. For more information on segmentation, please read the segmentation overview.

More help on this topic

- Miscellaneous functions
- Logical quantifier functions
- Supported data types in Segmentation Service

Previous page
ENFORCE DATA USAGE COMPLIANCE FOR AUDIENCES

Next page
BOOLEAN FUNCTIONS